FastMatMul: Efficient Verifiable Matrix Multiplication via Linear Error-Correcting Codes

Abstract

Verifiable machine learning inference requires proving that a claimed output was computed correctly, yet the dominant computational primitive—matrix multiplication—creates a severe bottleneck for SNARK-based approaches: naïve arithmetization of a $k \times k$ product incurs $O(k^3)$ constraints, and even the classical Freivalds reduction yields $O(k^2)$.

We present FastMatMul, a verifiable computation protocol that reduces the SNARK circuit size for matrix multiplication to $O(tk)$, where t is a small security-parameter-dependent constant independent of k. The protocol combines a structured variant of Freivalds' randomized check with *proximity testing over linear error-correcting codes*: matrices are encoded row-wise and committed via Merkle trees; the verifier detects any inconsistency by querying t random codeword positions and checking lightweight algebraic fold relations. The expensive $O(k^3)$ matrix arithmetic is performed entirely outside the SNARK. We prove that the protocol achieves perfect completeness and computational soundness with error $\epsilon_{\text{bind}} + (k-1)/|\mathbb{F}| + 4(1-\delta)^t$, where δ is the relative distance of the underlying code.

We implement FastMatMul in Rust and evaluate it against a direct Freivalds SNARK baseline. Experiments show a 30× constraint reduction at $k = 4{,}096$ and 6.6× proving-time speedup, while the baseline runs out of memory at $k = 8{,}192$ and FastMatMul continues to scale. The proving-time advantage materializes for $k \geq 1{,}024$—the regime relevant to Transformer attention layers and feed-forward projections in modern large language models.

ACM Reference Format:

. 2025. FastMatMul: Efficient Verifiable Matrix Multiplication via Linear Error-Correcting Codes. In *Proceedings of ACM Conference on Computer and Communications Security (CCS '25).* ACM, New York, NY, USA, 12 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

The deployment of machine learning models in outsourced and multi-party settings has created an urgent need for *verifiable inference*: a protocol in which a prover (e.g., a cloud GPU server) convinces a verifier (e.g., a client or smart contract) that a claimed output was computed correctly from a given model and input. Zero-Knowledge Succinct Non-interactive Arguments of Knowledge (zk-SNARKs) are the natural cryptographic tool for this task, but their application to modern deep-learning architectures—particularly Transformer-based large language models—has been hampered by a single, dominant bottleneck: *matrix multiplication*. In a standard Transformer layer, fully-connected projections and attention computations are expressed as products of matrices with dimensions $k \geq 1{,}000$. Naïve SNARK arithmetization of a single $k \times k$ matrix multiplication generates $O(k^3)$ arithmetic constraints, a cost that is impractical even for moderate k.

The $O(k^2)$ barrier. Freivalds' classical randomized check [8] reduces the problem from $O(k^3)$ to $O(k^2)$: given matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ and a random vector $\mathbf{r}$, one verifies $\mathbf{rAB} = \mathbf{rC}$ instead of recomputing the full product. Several recent verifiable ML systems—Mystique [18], zkML [4], and the Freivalds-based baseline in [9]—adopt this strategy, encoding the vector-matrix products directly as SNARK constraints. The resulting $O(k^2)$ circuit is an improvement, yet it remains expensive: at $k = 4{,}096$, the direct Freivalds arithmetization requires over 50 million R1CS constraints, takes 773 s to prove, and exceeds available memory at $k = 8{,}192$. Since modern Transformer attention layers routinely operate at $k \geq 1{,}024$, the gap between the $O(k^2)$ SNARK cost and practical deployability remains wide.

Our insight: from computing to checking. We observe that the $O(k^2)$ barrier is *not* inherent to the verification problem—it arises because existing approaches *compute* the vector-matrix products inside the SNARK circuit. Our key idea is to shift the paradigm from "computing in the circuit" to "*checking* in the circuit": the prover performs the expensive matrix arithmetic *outside* the SNARK and commits to all intermediate results; the verifier then checks a small number of lightweight algebraic relations that suffice to detect any inconsistency.

The technical engine behind this shift is *proximity testing over linear error-correcting codes.* By encoding each matrix row-wise with a systematic linear code C of relative distance δ, the vector-matrix product $\mathbf{x} = \mathbf{rA}$ can be verified *column by column* in the encoded domain: the linearity of C ensures that a single inner product ("fold") of one codeword column suffices to check one coordinate of the encoded product. If the prover cheats on any relation, the committed codewords disagree in at least a δ-fraction of positions, and a random query detects the inconsistency with probability $\geq \delta$. Crucially, the SNARK circuit only encodes the *local checks*—fold computations and Merkle path verifications at t queried positions—yielding a circuit size of $O(tk)$, which is *linear* in the matrix dimension when t is a small security-parameter-dependent constant.

Contributions. We present FastMatMul, a verifiable computation protocol for matrix multiplication that realizes this approach. Our contributions are as follows.

- **A novel code-based verification protocol.** We design a three-round public-coin protocol that combines a structured variant of Freivalds' randomized reduction with proximity testing over linear codes and Merkle-tree commitments. When instantiated with linear-time encodable codes (e.g.,

CCS '25, Salt Lake City, UT, USA

ACM ISBN 978-1-4503-XXXX-X/2025/10
https://doi.org/XXXXXXX.XXXXXXX

expander-based codes [11]), the resulting SNARK circuit size is $O(tk)$—a significant reduction from the $O(k^2)$ of Freivalds-based approaches. For general constant-rate codes, encoding constraints contribute an additional $O(k^2)$ term; we analyze this trade-off in Section 4.3. The protocol is made non-interactive via the Fiat–Shamir heuristic [7].

- **Rigorous security analysis.** We prove that FastMatMul achieves perfect completeness and computational soundness with error bounded by $\epsilon_{\text{bind}} + (k-1)/|\mathbb{F}| + 4(1-\delta)^t$, where δ is the code's relative distance and ϵ_{bind} is the Merkle commitment's binding error. The analysis decomposes the adversary's strategy into three orthogonal failure modes—commitment forgery, algebraic evasion, and proximity evasion—and shows each is negligible for standard parameter choices (Section 5).
- **Concrete efficiency gains.** We implement FastMatMul in Rust and evaluate it against a direct Freivalds SNARK baseline across matrix dimensions $k \in [2^5, 2^{13}]$. Key findings (Section 6):
  - *Constraint reduction.* FastMatMul constraints grow as $O(k)$, versus $O(k^2)$ for Freivalds. The gap reaches 30× at k = 4,096 (1,664,106 vs. 50,491,721 constraints); Freivalds becomes infeasible at k = 8,192 while FastMatMul completes with 3,324,484 constraints.
  - *Proving-time speedup.* FastMatMul outperforms Freivalds for $k \geq$ 1,024, achieving 1.7×, 2.7×, and 6.6× speedups at k = 1,024, 2,048, and 4,096 respectively. At k = 8,192, FastMatMul proves in 466 s; Freivalds cannot run.
  - *Negligible encoding overhead.* The prover's off-circuit encoding step takes less than 1 s even at $k = 2^{20}$, confirming that encoding does not introduce a new bottleneck.

Paper organization. Section 2 gives a high-level technical overview. Section 3 recalls the necessary background. Section 4 presents the formal construction and complexity analysis. Section 5 provides the security proof. Section 6 reports our implementation and evaluation. Section 7 surveys related work, and Section 8 concludes with future directions.

2 Technical Overview

We provide a high-level description of our protocol for verifiable matrix multiplication before presenting the formal construction in Section 4. The central challenge is to verify that a claimed product $\mathbf{C} = \mathbf{AB}$ for matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k\times k}$ is correct, while keeping the verifier's work substantially below the $O(k^3)$ cost of recomputing the multiplication. Our approach combines three classical ideas—*randomized verification*, *error-correcting codes*, and *succinct commitment schemes*—in a way that decomposes the verification into components that are each efficient to check inside a SNARK circuit.

2.1 Starting Point: Freivalds' Algorithm and Its Limitations

The classical starting point for verifying matrix multiplication is Freivalds' algorithm [8]. Given matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k\times k}$ and a uniformly random vector $\mathbf{r} \in \mathbb{F}^k$, one checks whether $\mathbf{rAB} = \mathbf{rC}$. If $\mathbf{AB} \neq \mathbf{C}$, then $\mathbf{D} \coloneqq \mathbf{AB} - \mathbf{C} \neq \mathbf{0}$, and the check $\mathbf{rD} = \mathbf{0}$ fails with probability at least $1 - k/|\mathbb{F}|$ by the Schwartz–Zippel lemma. This reduces a matrix-matrix verification to a vector-matrix computation, collapsing the problem from $O(k^3)$ to $O(k^2)$.

However, even the reduced $O(k^2)$ cost is prohibitive when the check must be performed *inside a SNARK circuit*. In an R1CS or Plonk arithmetization, each field multiplication corresponds to a constraint, so encoding the vector-matrix products $\mathbf{rA}$, $\mathbf{rB}$, and $\mathbf{rC}$ directly would still produce $O(k^2)$ constraints. For matrices arising in modern deep-learning inference (where k can reach thousands), this SNARK circuit cost remains impractical.

2.2 From Vectors to Codewords: Leveraging Linear Codes

Our key observation is that the *linearity* of error-correcting codes can be exploited to push the bulk of the computation outside the circuit while retaining verifiability. Concretely, suppose we encode each row of a matrix $\mathbf{M} \in \mathbb{F}^{k\times k}$ using a systematic linear code $C : \mathbb{F}^k \to \mathbb{F}^n$ with relative minimum distance δ. Write $\mathsf{Enc}(\mathbf{M})$ for the matrix whose i-th row is $C(\mathbf{M}_{i,\star})$. Because C is $\mathbb{F}$-linear, taking a linear combination of codewords yields the codeword of the corresponding linear combination of messages:

$$\sum_i r_i \cdot C(\mathbf{M}_{i,\star}) = C\left(\sum_i r_i \cdot \mathbf{M}_{i,\star}\right). \tag{1}$$

In other words, the "fold" operation $\mathsf{Fold}(\mathsf{Enc}(\mathbf{M}), \mathbf{r}) \coloneqq \sum_i r_i \cdot \mathsf{Enc}(\mathbf{M})_{i,\star}$ equals $\mathsf{Enc}(\mathbf{rM})$ exactly. This identity is the engine that allows us to *check vector-matrix products column-by-column in the encoded domain* without materializing the full vectors.

Commitment via Merkle trees. Each encoded matrix $\mathsf{Enc}(\mathbf{M})$ is committed column-by-column using a Merkle tree: $\mathsf{cm}_M \leftarrow \mathsf{M.CM}(\mathsf{Enc}(\mathbf{M}))$. The prover sends only the root hashes $(\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C)$ to the verifier, binding the computation inputs and output at the cost of $O(1)$ hash values.

2.3 Proximity-Based Verification via Random Sampling

After the Freivalds reduction, the verification task becomes: given committed encoded matrices $\mathsf{Enc}(\mathbf{A})$, $\mathsf{Enc}(\mathbf{B})$, $\mathsf{Enc}(\mathbf{C})$ and a random challenge r, check that the intermediate vectors

$$\mathbf{x} = \mathbf{rA}, \quad \mathbf{y} = \mathbf{xB}, \quad \mathbf{z} = \mathbf{rC}$$

satisfy $\mathbf{y} = \mathbf{z}$. We now explain how to verify these relations in *sub-linear* time using the encoding.

The prover computes $\mathbf{x}, \mathbf{y}, \mathbf{z}$ outside the SNARK, encodes them, and commits to their codewords $(\mathsf{cm}_x, \mathsf{cm}_y, \mathsf{cm}_z)$. The verifier then selects a small random subset of column indices $I \subset [n]$ with $|I| = t$, and for each $j \in I$ requests the j-th column of every committed codeword together with its Merkle authentication path.

At each queried position j, the verifier checks four *local consistency equations*:

(i) $\mathsf{Enc}(\mathbf{x})_j = \mathsf{Fold}\big(\mathsf{Enc}(\mathbf{A})^{(j)}, \mathbf{r}\big)$, ensuring $\mathbf{x} = \mathbf{rA}$;
(ii) $\mathsf{Enc}(\mathbf{y})_j = \mathsf{Fold}\big(\mathsf{Enc}(\mathbf{B})^{(j)}, \mathbf{x}\big)$, ensuring $\mathbf{y} = \mathbf{xB}$;
(iii) $\mathsf{Enc}(\mathbf{z})_j = \mathsf{Fold}\big(\mathsf{Enc}(\mathbf{C})^{(j)}, \mathbf{r}\big)$, ensuring $\mathbf{z} = \mathbf{rC}$;
(iv) $\mathsf{Enc}(\mathbf{y})_j = \mathsf{Enc}(\mathbf{z})_j$, ensuring the Freivalds check $\mathbf{y} = \mathbf{z}$.

The crucial point is that each of these checks involves only a *single column* of the encoded matrices—an inner product of length k (for the fold operations) plus a scalar comparison. Hence the total verifier work per query is $O(k)$, and for t queries the total is $O(tk)$.

Why does sampling suffice? If any of the underlying vector relations fails—say $\mathbf{y} \neq \mathbf{z}$—then the codewords $\mathsf{Enc}(\mathbf{y})$ and $\mathsf{Enc}(\mathbf{z})$ are distinct. By the minimum-distance property of C, they disagree in at least a δ-fraction of positions. A uniformly random query lands on a disagreement with probability $\geq \delta$, so t independent queries detect the inconsistency except with probability at most $(1-\delta)^t$. Setting $t = O(\lambda/\log(1/(1-\delta)))$ drives the soundness error below $2^{-\lambda}$. By a union bound over the four checked relations, the overall proximity-check failure probability is at most $4(1-\delta)^t$.

2.4 Putting It Together: The Full Protocol Pipeline

Combining the ingredients above, our protocol FastMatMul proceeds as follows (see Figure 1 for the formal description).

Phase 1: Commit. The prover encodes $\mathbf{A}, \mathbf{B}, \mathbf{C}$ row-wise using the linear code C, commits to the resulting codeword matrices via Merkle trees, and sends roots $(\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C)$ to the verifier.

Phase 2: Challenge. The verifier samples a random field element $r \in \mathbb{F}$ and sends it to the prover. This induces the structured challenge vector $\mathbf{r} = (1, r, r^2, \ldots, r^{k-1})$.

Phase 3: Respond. The prover computes the intermediate vectors $\mathbf{x} = \mathbf{rA}, \mathbf{y} = \mathbf{xB}, \mathbf{z} = \mathbf{rC}$, encodes them, and commits to the encodings $(\mathsf{cm}_x, \mathsf{cm}_y, \mathsf{cm}_z)$.

Phase 4: Query. The verifier samples random column indices $I \subset [n]$ and sends them to the prover. The prover opens the corresponding columns of all six committed codewords along with their Merkle proofs.

Phase 5: Verify (in SNARK).. The verifier checks Merkle path validity and the four local consistency equations at each queried index. Crucially, these checks are *arithmetized* into a SNARK relation $\mathcal{R}_{\mathsf{SNARK}}$ (defined formally in Section 4), and the prover produces a succinct proof π attesting that all checks pass. The verifier only needs to verify π, reducing the total verifier cost to that of a single SNARK verification.

Cost summary. The *SNARK circuit size*—the key metric governing prover cost—is $O(t \cdot k)$, linear in k and independent of k^2 or k^3. The prover additionally performs the native matrix multiplication ($O(k^3)$) and encoding ($O(kn)$) outside the SNARK. The verifier checks only the SNARK proof: $O(1)$ for pairing-based backends, or $O(|\mathsf{x}|)$ for transparent ones. A detailed cost breakdown appears in Table 2 (Section 4.3).

3 Preliminaries

We introduce notation and recall the building blocks used in our construction. Throughout, $\mathbb{F}$ denotes a finite field of size $|\mathbb{F}| = p$ for a prime p, and λ denotes the security parameter. We write $\mathsf{negl}(\lambda)$ for any function that is negligible in λ, and PPT for probabilistic polynomial-time.

3.1 Notation

Matrices are denoted by bold uppercase letters $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}$ and vectors by bold lowercase letters $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{F}^k$. For a matrix $\mathbf{M}$, we write $\mathbf{M}_{i,\star}$ for its i-th row and $\mathbf{M}_{\star,j}$ for its j-th column, both viewed as elements of $\mathbb{F}^k$. We write $[n] = \{1, 2, \ldots, n\}$ and $I \leftarrow\!\$\ [n]^t$ to denote the uniform sampling of t indices from $[n]$ (with replacement). The expression $r \leftarrow\!\$\ \mathbb{F}$ denotes uniform random sampling from $\mathbb{F}$.

3.2 Linear Error-Correcting Codes

A *linear code* over $\mathbb{F}$ is an $\mathbb{F}$-linear map $C : \mathbb{F}^k \to \mathbb{F}^n$ defined by a generator matrix $G \in \mathbb{F}^{k \times n}$, so that $C(\mathbf{m}) = \mathbf{m} \cdot G$ for any message $\mathbf{m} \in \mathbb{F}^k$. The code has *rate* $\rho = k/n$ and *minimum distance* $d = \min_{\mathbf{c} \neq \mathbf{c}'} \Delta(\mathbf{c}, \mathbf{c}')$ over distinct codewords $\mathbf{c}, \mathbf{c}' \in C$, where Δ denotes the Hamming distance. The *relative distance* is $\delta = d/n$. A code is *systematic* if the first k coordinates of $C(\mathbf{m})$ equal $\mathbf{m}$ itself.

We write $\mathsf{Enc}(\mathbf{m}) \coloneqq C(\mathbf{m})$ for the encoding of a message vector. For a matrix $\mathbf{M} \in \mathbb{F}^{k \times k}$, we define the *row-wise encoding* $\mathsf{Enc}(\mathbf{M}) \in \mathbb{F}^{k \times n}$ as the matrix whose i-th row is $\mathsf{Enc}(\mathbf{M}_{i,\star})$. The j-th column of $\mathsf{Enc}(\mathbf{M})$ is denoted $\mathsf{Enc}(\mathbf{M})^{(j)} \in \mathbb{F}^k$.

The key property we exploit is *linearity*: for any coefficients $r_0, \ldots, r_{k-1} \in \mathbb{F}$,

$$\sum_{i=0}^{k-1} r_i \cdot \mathsf{Enc}(\mathbf{M}_{i,\star}) = \mathsf{Enc}\left(\sum_{i=0}^{k-1} r_i \cdot \mathbf{M}_{i,\star}\right). \tag{2}$$

Definition 3.1 (Fold operation). For a column vector $\mathbf{c} = (c_0, \ldots, c_{k-1})^\top \in \mathbb{F}^k$ and a coefficient vector $\mathbf{r} = (r_0, \ldots, r_{k-1}) \in \mathbb{F}^k$, we define

$$\mathsf{Fold}(\mathbf{c}, \mathbf{r}) \coloneqq \langle \mathbf{r}, \mathbf{c} \rangle = \sum_{i=0}^{k-1} r_i \cdot c_i.$$

Equivalently, for the j-th column of a row-wise encoded matrix, $\mathsf{Fold}(\mathsf{Enc}(\mathbf{M})^{(j)}, \mathbf{r})$ equals the j-th coordinate of $\mathsf{Enc}(\mathbf{rM})$. This follows directly from the linearity of the code (Equation 2).

Concrete instantiations. Our protocol is compatible with any linear code. Two natural choices are: (i) *Reed–Solomon codes*, where $C(\mathbf{m})$ evaluates the polynomial $\sum_i m_i X^i$ over a domain of size $n > k$, achieving the optimal (MDS) distance $\delta = 1 - k/n + 1/n$; and (ii) *expander-based codes* as used in Brakedown [11], which achieve linear-time encoding $O(n)$ with constant relative distance. We discuss the trade-offs in Section 4.3.

3.3 Merkle Tree Commitment

A *Merkle tree* [15] over a collision-resistant hash function $H : \{0,1\}^* \to \{0,1\}^\lambda$ provides a vector commitment scheme with the following algorithms:

- $\mathsf{cm} \leftarrow \mathsf{M.CM}(\mathbf{v})$: Given a vector $\mathbf{v} = (v_1, \ldots, v_n)$, construct a binary hash tree with leaves $H(v_1), \ldots, H(v_n)$ and output the root hash cm.
- $(\mathbf{v}_j, \pi_j) \leftarrow \mathsf{Merkle.Open}(\mathsf{cm}, j)$: Open position j, returning the leaf value v_j and the authentication path π_j (the sequence of sibling hashes from the leaf to the root).
- $b \leftarrow \mathsf{Merkle.Verify}(\pi_j, v_j, \mathsf{cm})$: Verify that v_j is the j-th leaf consistent with root cm; output $b \in \{0,1\}$.

The scheme is *position-binding*: under the collision resistance of H, no PPT adversary can produce (j, v, π) and (j, v', π') with

$v \neq v'$ such that both verify against the same root cm, except with probability $\epsilon_{\mathsf{bind}}(\lambda) = \mathsf{negl}(\lambda)$.

3.4 Succinct Non-Interactive Arguments of Knowledge

A *SNARK* (Succinct Non-interactive Argument of Knowledge) for an NP relation $\mathcal{R}$ consists of algorithms (Setup, SNARK.P, SNARK.V):

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{R})$: Generate public parameters.
- $\pi \leftarrow \mathsf{SNARK.P}(\mathsf{pp}, \mathsf{x}, \mathsf{w})$: Given a statement x and witness w with $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}$, produce a proof π.
- $b \leftarrow \mathsf{SNARK.V}(\mathsf{pp}, \mathsf{x}, \pi)$: Verify the proof and output $b \in \{0, 1\}$.

A SNARK satisfies *completeness* (an honest prover always convinces the verifier), *knowledge soundness* (any PPT prover that makes the verifier accept can be used to extract a valid witness), and *succinctness* (the proof size $|\pi|$ and verifier time are $\mathsf{poly}(\lambda, |\mathsf{x}|)$, independent of the witness size and computation).

Our protocol is modular with respect to the choice of SNARK backend. The key metric is the *circuit size* (number of R1CS constraints or PlonK gates) of the relation $\mathcal{R}_{\mathsf{SNARK}}$ that the backend must prove, as this dominates the prover's cost.

3.5 Freivalds' Algorithm

Freivalds' algorithm [8] is a classical randomized algorithm for verifying matrix multiplication. Given $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}$, it samples $\mathbf{r} \leftarrow\!\!\$\ \mathbb{F}^k$ and checks whether $\mathbf{r}(\mathbf{AB}) = \mathbf{rC}$. The correctness relies on the following:

Lemma 3.2 (Schwartz–Zippel [17, 21]). *Let $P(X_1, \ldots, X_m)$ be a non-zero polynomial of total degree d over a finite field $\mathbb{F}$. Then for $r_1, \ldots, r_m \leftarrow\!\!\$\ \mathbb{F}$ chosen uniformly and independently,*

$$\Pr[P(r_1, \ldots, r_m) = 0] \leq \frac{d}{|\mathbb{F}|}.$$

In our protocol, we use a structured variant: instead of sampling $\mathbf{r} \leftarrow\!\!\$\ \mathbb{F}^k$ uniformly, we sample a single $r \leftarrow\!\!\$\ \mathbb{F}$ and set $\mathbf{r} = (1, r, r^2, \ldots, r^{k-1})$. If $\mathbf{D} = \mathbf{AB} - \mathbf{C} \neq \mathbf{0}$, then for any non-zero column $\mathbf{d}_j$ of $\mathbf{D}$, the inner product $\langle \mathbf{r}, \mathbf{d}_j \rangle = \sum_{i=0}^{k-1} (\mathbf{D})_{i,j} \cdot r^i$ is a non-zero univariate polynomial in r of degree at most $k-1$. By Lemma 3.2, this evaluates to zero with probability at most $(k-1)/|\mathbb{F}|$.

The Vandermonde structure offers a concrete efficiency advantage beyond mere randomness reduction. The challenge r appears in the SNARK public statement x (Section 4); using a single scalar keeps $|\mathsf{x}| = O(1)$ field elements, independent of k. Had we used a uniformly random $\mathbf{r} \in \mathbb{F}^k$, the statement size would grow to $O(k)$, increasing verifier time for statement-size-sensitive SNARKs (e.g., transparent SNARKs where verifier time is $O(|\mathsf{x}|)$) and defeating the goal of a k-independent verifier cost. The soundness bound $(k-1)/|\mathbb{F}|$ is slightly weaker than the $1/|\mathbb{F}|$ bound of a fully random $\mathbf{r}$, but remains negligible for any $|\mathbb{F}| \geq 2^{256}$ and practical k.

4 Our Construction

In this section we present the formal construction of our verifiable matrix multiplication protocol FastMatMul. We begin by defining the relation chain that reduces the original matrix multiplication statement to a succinct SNARK-friendly relation (Section 4.1), then give the full protocol description (Section 4.2), and conclude with a complexity analysis (Section 4.3). Table 1 summarizes the notation used throughout the paper.

Table 1: Summary of notation.

Symbol	Description
Parameters	
$\mathbb{F}$	Finite field
k	Matrix dimension ($k \times k$)
n	Codeword length of C
δ	Relative minimum distance of C
$t = \lvert I \rvert$	Query complexity (number of sampled indices)
λ	Security parameter
Matrices and Vectors	
$\mathbf{A}, \mathbf{B}, \mathbf{C}$	Input/output matrices in $\mathbb{F}^{k \times k}$
$\mathbf{D}$	Error matrix $\mathbf{AB} - \mathbf{C}$
$\mathbf{r}$	Challenge vector $(1, r, r^2, \ldots, r^{k-1})$
$\mathbf{x}, \mathbf{y}, \mathbf{z}$	Intermediate vectors: $\mathbf{rA}$, $\mathbf{xB}$, $\mathbf{rC}$
Encoding and Commitment	
C	Linear error-correcting code $\mathbb{F}^k \to \mathbb{F}^n$
$\mathsf{Enc}(\cdot)$	Encoding function (row-wise for matrices)
$\mathsf{Enc}(\mathbf{M})^{(j)}$	j-th column of $\mathsf{Enc}(\mathbf{M}) \in \mathbb{F}^{k \times n}$
$\mathsf{Fold}(\mathbf{c}, \mathbf{r})$	Inner product $\langle \mathbf{r}, \mathbf{c} \rangle$
$\mathsf{M.CM}(\cdot)$	Merkle tree commitment
$\mathsf{Merkle.Open}(\mathsf{cm}, j)$	Merkle tree opening at index j
$\mathsf{Merkle.Verify}(\pi, v, \mathsf{cm})$	Merkle tree verification
cm_χ	Merkle root for committed object χ
$\pi_{\chi,j}$	Merkle authentication path for χ at position j
Sets	
$\mathcal{M}$	Matrix set $\{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$
$\mathcal{W}$	Vector set $\{\mathbf{x}, \mathbf{y}, \mathbf{z}\}$
$\mathcal{S}$	Full committed-object set $\mathcal{M} \cup \mathcal{W}$
I	Set of queried column indices ($I \subset [n]$)
SNARK	
x	SNARK public statement (instance)
w	SNARK witness
π	SNARK proof
SNARK.P, SNARK.V	SNARK prover and verifier
pp	Public parameters

4.1 Relation Reduction Chain

Our construction proceeds through two successive reductions. Each reduction preserves completeness exactly and introduces a bounded soundness error that we analyze in Section 5.

The original relation $\mathcal{R}_{\mathsf{orig}}$. The starting point is the straightforward statement that the prover knows matrices whose committed values satisfy the multiplication relation. The instance is $(\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C)$ and the witness is $(\mathbf{A}, \mathbf{B}, \mathbf{C})$; the relation $\mathcal{R}_{\mathsf{orig}}$ requires:

$$\mathsf{cm}_A = \mathsf{M.CM}(\mathbf{A}) \ \wedge\ \mathsf{cm}_B = \mathsf{M.CM}(\mathbf{B}) \ \wedge\ \mathsf{cm}_C = \mathsf{M.CM}(\mathbf{C}),$$
$$\mathbf{AB} = \mathbf{C}.$$

Proving membership in $\mathcal{R}_{\mathsf{orig}}$ directly inside a SNARK would require $O(k^3)$ multiplication constraints, which is exactly what we aim to avoid.

Reduction 1: Freivalds' randomization ($\mathcal{R}_{\text{orig}} \rightsquigarrow \mathcal{R}_{\text{Freivalds}}$). Using the verifier's random challenge $r \in \mathbb{F}$, we construct the vector $\mathbf{r} = (1, r, r^2, \ldots, r^{k-1})$ and define intermediate vectors $\mathbf{x} = \mathbf{rA}$, $\mathbf{y} = \mathbf{xB}$, $\mathbf{z} = \mathbf{rC}$. Let $\mathcal{S} = \{\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{x}, \mathbf{y}, \mathbf{z}\}$ denote the full set of committed objects. The instance is $(\{\mathsf{cm}_\chi\}_{\chi\in\mathcal{S}}, r)$ and the witness is $\{\chi\}_{\chi\in\mathcal{S}}$; the relation $\mathcal{R}_{\text{Freivalds}}$ requires:

$$\bigwedge_{\chi\in\mathcal{S}} \mathsf{cm}_\chi = \mathsf{M.CM}(\chi),$$
$$\mathbf{rA} = \mathbf{x} \ \wedge \ \mathbf{xB} = \mathbf{y},$$
$$\mathbf{rC} = \mathbf{z} \ \wedge \ \mathbf{y} = \mathbf{z}.$$

By the Schwartz–Zippel lemma, if $\mathbf{AB} \neq \mathbf{C}$, then $\mathbf{y} \neq \mathbf{z}$ with probability at least $1-(k-1)/|\mathbb{F}|$. Note that while this relation eliminates the $O(k^3)$ matrix product, verifying the vector-matrix products still costs $O(k^2)$ each.

Reduction 2: Proximity lifting ($\mathcal{R}_{\text{Freivalds}} \rightsquigarrow \mathcal{R}_{\text{SNARK}}$). We now encode all matrices and vectors using a systematic $\mathbb{F}$-linear code $C : \mathbb{F}^k \to \mathbb{F}^n$ with relative distance δ, and replace the global vector-matrix checks with *local* consistency checks at random positions. Let $I \subset [n]$ be the set of queried column indices, and let $\mathcal{M} = \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}, \mathcal{W} = \{\mathbf{x}, \mathbf{y}, \mathbf{z}\}$ denote the matrix set and vector set respectively.

The *public instance* is $\mathsf{x} = \big(\{\mathsf{cm}_\chi\}_{\chi\in\mathcal{S}},\ r,\ I\big)$ and the *witness* is

$$\mathsf{w} = \Big(\mathbf{x},\ \mathbf{y},\ \mathbf{z},\ \big\{\mathsf{Enc}_\chi^{(j)},\ \pi_{\chi,j}\big\}_{j\in I,\,\chi\in\mathcal{S}}\Big).$$

The relation $\mathcal{R}_{\text{SNARK}}$ requires (x, w) to satisfy all of:

$$\mathsf{Enc}(\mathbf{x}) = \mathsf{Enc}_x \ \wedge \ \mathsf{Enc}(\mathbf{y}) = \mathsf{Enc}_y \ \wedge \ \mathsf{Enc}(\mathbf{z}) = \mathsf{Enc}_z, \tag{ENC}$$

$$\forall j \in I: \quad \mathsf{Enc}_{x,j} = \mathsf{Fold}\big(\mathsf{Enc}_A^{(j)},\ \mathbf{r}\big), \tag{FOLD-A}$$

$$\forall j \in I: \quad \mathsf{Enc}_{y,j} = \mathsf{Fold}\big(\mathsf{Enc}_B^{(j)},\ \mathbf{x}\big), \tag{FOLD-B}$$

$$\forall j \in I: \quad \mathsf{Enc}_{z,j} = \mathsf{Fold}\big(\mathsf{Enc}_C^{(j)},\ \mathbf{r}\big), \tag{FOLD-C}$$

$$\forall j \in I: \quad \mathsf{Enc}_{y,j} = \mathsf{Enc}_{z,j}, \tag{EQ}$$

$$\forall \chi \in \mathcal{S},\ j \in I: \quad \mathsf{Merkle.Verify}\big(\pi_{\chi,j},\ \mathsf{Enc}_\chi^{(j)},\ \mathsf{cm}_\chi\big) = 1. \tag{PATH}$$

The critical property is that $\mathcal{R}_{\text{SNARK}}$ involves only $O(|I| \cdot k)$ constraints for the fold operations, plus $O(|I| \cdot \log n)$ hash checks for the Merkle paths. Since $|I|$ is a security-parameter-sized constant independent of k, the circuit size is *linear in* k.

Encoding constraints. The constraints $\mathsf{Enc}(v) = \mathsf{Enc}_v$ for $v \in \mathcal{W}$ force the prover to commit to *valid codewords*. Without them, a cheating prover could commit to arbitrary strings that happen to satisfy the fold checks at queried positions without corresponding to any genuine encoding, defeating the distance-based soundness argument. We note that for a linear code with generator matrix $G \in \mathbb{F}^{k\times n}$, the constraint $\mathsf{Enc}_v = v \cdot G$ requires $O(k(n-k))$ inner-product checks. For codes with linear-time encoding (e.g., expander-based codes [11]), this is $O(kn)$ with small constants; for constant-rate codes $n = O(k)$, the encoding constraints contribute $O(k^2)$ total. We discuss the trade-off in Section 4.3.

4.2 Protocol Description

We now give the full specification of FastMatMul. Let $C : \mathbb{F}^k \to \mathbb{F}^n$ be a systematic linear code with relative distance δ and M.CM denote the Merkle tree commitment. Let $\mathcal{M} = \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$ be the matrix set, $\mathcal{W} = \{\mathbf{x}, \mathbf{y}, \mathbf{z}\}$ the vector set, and $\mathcal{S} = \mathcal{M} \cup \mathcal{W}$. The protocol FastMatMul operates between a prover $\mathcal{P}$ and verifier $\mathcal{V}$ as shown in Figure 1.

Remark on the SNARK relation. The SNARK proof π attests to the validity of the relation $\mathcal{R}_{\text{SNARK}}$ defined in Section 4.1. The SNARK circuit verifies: (1) constraint (ENC), ensuring $\mathbf{x}, \mathbf{y}, \mathbf{z}$ are valid codewords of C; (2) constraints (FOLD-A)–(EQ), the four fold/equality checks at each $j \in I$; and (3) constraint (PATH), consistency of all Merkle opening proofs with the committed roots. The public statement x consists of the commitment roots $\{\mathsf{cm}_\chi\}_{\chi\in\mathcal{S}}$, the challenge r, and the query set I. The witness w consists of the intermediate vectors and the opened codeword columns with their authentication paths.

Non-interactive variant. The protocol as described is a three-round public-coin interactive protocol. By applying the Fiat–Shamir heuristic [7] in the random oracle model, it can be made non-interactive: set $r \leftarrow \mathsf{Hash}(\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C)$ and $I \leftarrow \mathsf{Hash}(\mathsf{cm}_x, \mathsf{cm}_y, \mathsf{cm}_z)$. The resulting non-interactive proof consists of the six commitment roots together with the SNARK proof π.

Security of the non-interactive variant. A potential concern with the Fiat–Shamir transform for proximity-based protocols is a *grinding attack*: a malicious prover could try many different tuples $(\mathsf{cm}_x, \mathsf{cm}_y, \mathsf{cm}_z)$ until the resulting query set $I = \mathsf{Hash}(\mathsf{cm}_x, \mathsf{cm}_y, \mathsf{cm}_z)$ happens to avoid all positions where the committed codewords are inconsistent. We argue that this attack is infeasible in the random oracle model. Suppose the adversary has committed to $(\mathsf{cm}_A, \mathsf{cm}_B, \mathsf{cm}_C)$ (and hence to fixed encoded matrices) and is searching for $(\mathsf{cm}_x, \mathsf{cm}_y, \mathsf{cm}_z)$ such that I avoids the disagreement set D of some fold or equality check. If any of the four checks fails globally, then $|D| \geq \delta n$ by the minimum-distance property. For a single random query $j \in [n]$, the probability of $j \notin D$ is at most $1-\delta$; for t independent queries it is $(1-\delta)^t \leq 2^{-\lambda}$. Each trial of the grinding attack produces an independently random I in the random oracle model, so the probability that q grinding attempts all yield a favorable I is at most $q \cdot 2^{-\lambda}$. For any PPT adversary, q is polynomially bounded in λ, giving a total success probability of $\mathsf{negl}(\lambda)$. Hence the non-interactive protocol retains the soundness bound of Theorem 5.1 in the random oracle model.

Zero-knowledge extension. The protocol as stated is a *verifiable computation* protocol and does not conceal the inputs $\mathbf{A}, \mathbf{B}, \mathbf{C}$ from the verifier. Zero-knowledge can be achieved straightforwardly by two modifications: (i) replace the Merkle tree commitments with a *hiding* commitment scheme (e.g., salted hashes or Pedersen vector commitments [16]) so that the committed encoded matrices reveal no information about $\mathbf{A}, \mathbf{B}, \mathbf{C}$; and (ii) employ a zero-knowledge SNARK backend for the inner proof π. Masking the intermediate vectors $\mathbf{x}, \mathbf{y}, \mathbf{z}$ requires adding fresh random blinding vectors in Phase 3 and including the corresponding decommitment randomness in the SNARK witness. We omit the full ZK treatment here for clarity of presentation; the construction carries over to the ZK setting without structural changes to the protocol.

4.3 Complexity Analysis

We analyze the computational and communication costs of FastMatMul.

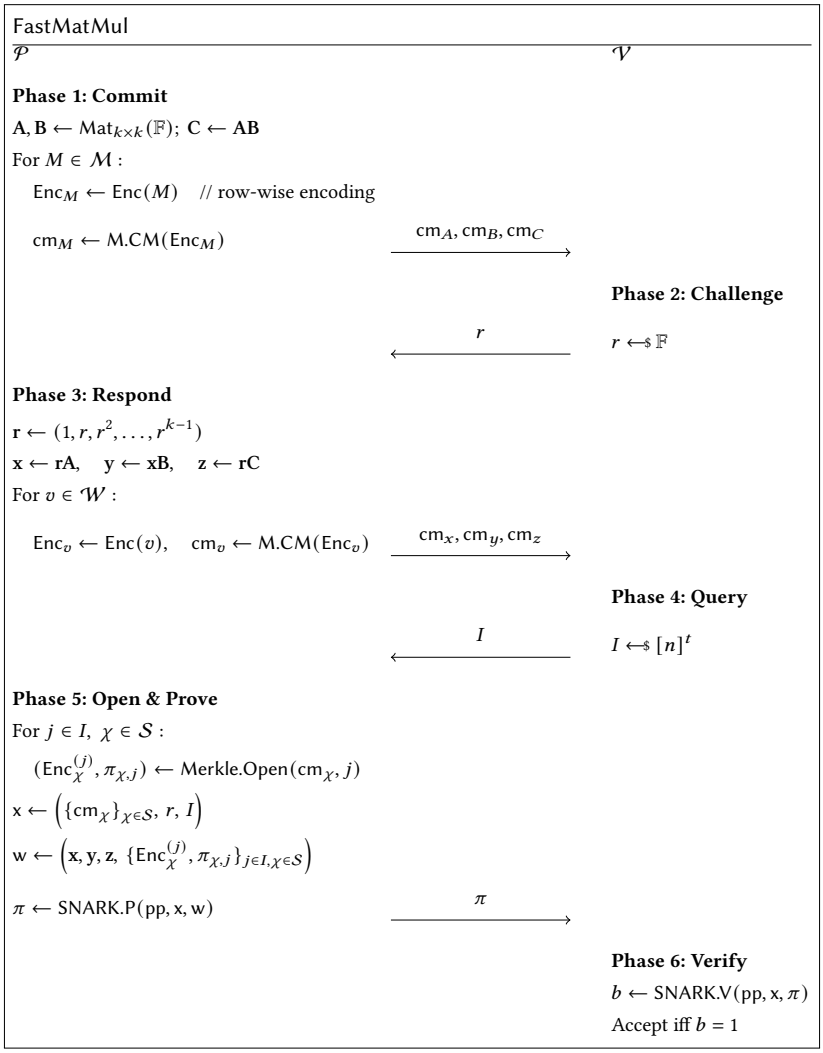

Figure 1: The FastMatMul protocol for verifiable matrix multiplication. $\mathcal{S} = \mathcal{M} \cup \mathcal{W}$ is the full set of committed objects, $t = |I|$ is the query complexity, and x, w denote the public statement and witness for the inner SNARK.

Prover computation. The prover's dominant costs are:

(i) *Matrix multiplication*: Computing $\mathbf{C} = \mathbf{AB}$ requires $O(k^3)$ field operations (or $O(k^\omega)$ with fast algorithms). This is performed natively outside the SNARK circuit and is amenable to GPU acceleration.
(ii) *Encoding*: $3k$ rows (one per matrix row, three matrices) each encoded into length-n codewords costs $O(kn)$ per matrix with linear-time codes.
(iii) *Vector computation*: Computing $\mathbf{x}, \mathbf{y}, \mathbf{z}$ costs $O(k^2)$ in total.
(iv) *SNARK proving*: The circuit size is $O(t \cdot k + t \cdot \log n)$ for the fold and Merkle checks, plus encoding constraints (see below). The SNARK prover time scales with the circuit size.

Verifier computation. The verifier performs only SNARK verification: $O(1)$ for pairing-based SNARKs, or $O(|\mathsf{x}|)$ for transparent SNARKs, both independent of the matrix dimension k.

Communication. The proof consists of six Merkle roots ($O(\lambda)$ bits) plus the SNARK proof π ($O(\lambda)$ bits for succinct schemes), giving total communication $O(\lambda)$.

Circuit size comparison. Table 2 summarizes the asymptotic costs. Naïve arithmetization of $\mathbf{AB} = \mathbf{C}$ requires $O(k^3)$ constraints. Freivalds' algorithm alone reduces this to $O(k^2)$. Our approach achieves $O(tk + k(n-k))$: the first term covers the fold and proximity checks; the second covers the encoding constraints. For constant-rate codes ($n = \Theta(k)$), the encoding constraints contribute $O(k^2)$, which can be comparable to the direct Freivalds cost—but with a much smaller constant factor and, crucially, no dependence on the prover's matrix output. With linear-time encodable codes of rate ρ and distance δ, setting $t = \lceil \lambda / \log_2(1/(1-\delta)) \rceil$ gives a circuit size of $O(k)$ (up to the encoding constraint term), with concrete sizes discussed in Section 6.

Table 2: Asymptotic cost comparison. k: matrix dimension; n: codeword length; t: query count; λ: security parameter.

Component	Prover	Verifier
Matrix multiplication	$O(k^3)$	—
Encoding (row-wise)	$O(kn)$	—
Merkle commitment	$O(n \log n)$	—
SNARK circuit size	$O(tk + k(n-k))$	—
SNARK verification	—	$O(1)$
Total	$O(k^3 + kn)$	$O(1)$

5 Security Analysis

We prove that the protocol FastMatMul (Figure 1) constitutes a secure interactive argument for the matrix multiplication relation $\mathcal{R}_{\text{orig}}$. Soundness and completeness are established in Theorem 5.1 below; knowledge soundness is discussed separately in Remark 1.

Theorem 5.1 (Security of FastMatMul). *The protocol FastMatMul satisfies the following properties:*

(i) ***Perfect completeness.*** *If* $\mathbf{AB} = \mathbf{C}$*, an honest prover always makes the verifier accept.*

(ii) ***Computational soundness.*** *For any* PPT *adversary* $\mathcal{P}^*$*, the probability that the verifier accepts a false statement* $\mathbf{AB} \neq \mathbf{C}$ *satisfies:*

$$\Pr[\mathcal{V} \text{ accepts} \mid \mathbf{AB} \neq \mathbf{C}] \;\le\; \epsilon_{\mathsf{bind}}(\lambda) + \frac{k-1}{|\mathbb{F}|} + 4(1-\delta)^t,$$

where $t = |I|$ *is the query complexity,* δ *is the relative minimum distance of* C*, and* $\epsilon_{\mathsf{bind}}(\lambda) = \mathsf{negl}(\lambda)$ *under collision-resistant hashing.*

Proof. We establish the two properties in turn.

Perfect completeness. Suppose the prover is honest and $\mathbf{AB} = \mathbf{C}$. Let $\mathbf{D} \coloneqq \mathbf{AB} - \mathbf{C} = \mathbf{0}$. For any challenge vector $\mathbf{r}$, we have $\mathbf{rD} = \mathbf{0}$, so the intermediate vectors satisfy:

$$\mathbf{y} = \mathbf{xB} = (\mathbf{rA})\mathbf{B} = \mathbf{r}(\mathbf{AB}) = \mathbf{rC} = \mathbf{z}.$$

The encoding constraints hold by construction: the honest prover sets $\mathsf{Enc}_v = \mathsf{Enc}(v)$ for each $v \in \mathcal{W}$. For the fold checks at any index $j \in [n]$, the linearity of C (Definition 3.1) gives:

$$\mathsf{Fold}(\mathsf{Enc}(\mathbf{A})^{(j)}, \mathbf{r}) = \mathsf{Enc}(\mathbf{rA})_j = \mathsf{Enc}_{x,j},$$

and likewise for $\mathbf{B}$ and $\mathbf{C}$. Since $\mathbf{y} = \mathbf{z}$, we also have $\mathsf{Enc}_{y,j} = \mathsf{Enc}_{z,j}$ for all j. All Merkle paths are valid by honest commitment. Therefore every check passes with probability 1, establishing perfect completeness.

Soundness decomposition. Let Acc denote the event that the verifier accepts, and suppose $\mathbf{AB} \neq \mathbf{C}$. We decompose the adversary's success probability via three bad events:

$$\begin{aligned}&\Pr[\mathsf{Acc} \mid \mathbf{AB} \neq \mathbf{C}]\\ &\le\; \Pr[\mathsf{Bad}_{\mathsf{bind}}] + \Pr\big[\mathsf{Bad}_{\mathsf{rand}} \mid \overline{\mathsf{Bad}_{\mathsf{bind}}}\big]\\ &\quad + \Pr\big[\mathsf{Bad}_{\mathsf{prox}} \mid \overline{\mathsf{Bad}_{\mathsf{bind}}} \wedge \overline{\mathsf{Bad}_{\mathsf{rand}}}\big].\end{aligned}$$

Step 1: Commitment binding. Define $\mathsf{Bad}_{\mathsf{bind}}$ as the event that $\mathcal{P}^*$ produces a valid Merkle opening for some index j inconsistent with the committed root—equivalently, finding a hash collision. Under collision resistance of the hash function:

$$\Pr[\mathsf{Bad}_{\mathsf{bind}}] \le \epsilon_{\mathsf{bind}}(\lambda) = \mathsf{negl}(\lambda).$$

Conditioned on $\overline{\mathsf{Bad}_{\mathsf{bind}}}$, every opened value is uniquely determined by the committed root. Hence the adversary commits to fixed matrices $\mathbf{A}, \mathbf{B}, \mathbf{C}$ and vectors $\mathbf{x}, \mathbf{y}, \mathbf{z}$ before seeing the verifier's challenge.

Step 2: Randomized reduction. Condition on $\overline{\mathsf{Bad}_{\mathsf{bind}}}$. The adversary has committed to fixed matrices with $\mathbf{AB} \neq \mathbf{C}$. Let $\mathbf{D} \coloneqq \mathbf{AB} - \mathbf{C} \neq \mathbf{0}$. Since $\mathbf{D} \neq \mathbf{0}$, there exists at least one column $\mathbf{d}_j$ of $\mathbf{D}$ with $\mathbf{d}_j \neq \mathbf{0}$.

The protocol requires $\mathbf{y} = \mathbf{z}$, i.e., $\mathbf{rD} = \mathbf{0}$. The inner product $\langle \mathbf{r}, \mathbf{d}_j \rangle = \sum_{i=0}^{k-1} (\mathbf{D})_{i,j} \cdot r^i$ defines a non-zero univariate polynomial $P_j(r)$ of degree at most $k-1$ in the random variable r. Since r is chosen after the adversary commits, by the Schwartz–Zippel lemma (Lemma 3.2):

$$\Pr[\mathsf{Bad}_{\mathsf{rand}}] = \Pr_{r \leftarrow \mathbb{F}}[\mathbf{rD} = \mathbf{0}] \le \Pr_{r \leftarrow \mathbb{F}}[P_j(r) = 0] \le \frac{k-1}{|\mathbb{F}|}.$$

Step 3: Proximity check. Condition on $\overline{\mathsf{Bad}_{\mathsf{bind}}} \wedge \overline{\mathsf{Bad}_{\mathsf{rand}}}$. The second condition implies that the committed vectors violate at least one of:

(a) $\mathbf{x} \neq \mathbf{rA}$, or (b) $\mathbf{y} \neq \mathbf{xB}$, or (c) $\mathbf{z} \neq \mathbf{rC}$, or (d) $\mathbf{y} \neq \mathbf{z}$.

The SNARK relation $\mathcal{R}_{\mathsf{SNARK}}$ (Section 4.1) includes the constraints $\mathsf{Enc}(v) = \mathsf{Enc}_v$ for each $v \in \mathcal{W}$, verified within the SNARK circuit. Conditioned on binding, the committed codeword Enc_v is therefore a genuine codeword of C.

Consider case (a): $\mathbf{x} \neq \mathbf{rA}$. Since C is injective, the codewords $\mathsf{Enc}(\mathbf{x})$ and $\mathsf{Enc}(\mathbf{rA})$ are distinct. By the minimum distance property of C:

$$\big|\{j \in [n] : \mathsf{Enc}(\mathbf{x})_j \neq \mathsf{Enc}(\mathbf{rA})_j\}\big| \ge \delta \cdot n.$$

By the linearity of C (Definition 3.1), $\mathsf{Fold}(\mathsf{Enc}(\mathbf{A})^{(j)}, \mathbf{r}) = \mathsf{Enc}(\mathbf{rA})_j$. The fold check $\mathsf{Enc}_{x,j} = \mathsf{Fold}(\mathsf{Enc}_A^{(j)}, \mathbf{r})$ fails whenever j falls into the disagreement set, which has size at least δn. A uniformly random query j avoids all disagreements with probability at most $1-\delta$, so t independent queries all pass with probability at most $(1-\delta)^t$. The same argument applies symmetrically to cases (b), (c), and (d). Applying the union bound over all four cases:

$$\Pr[\mathsf{Bad}_{\mathsf{prox}}] \le 4(1-\delta)^t.$$

Remark on the union bound factor. The factor of 4 is conservative. If the adversary commits to vectors satisfying cases (a)–(c) honestly (i.e., $\mathbf{x} = \mathbf{rA}$, etc.), then $\mathbf{AB} \neq \mathbf{C}$ forces $\mathbf{y} \neq \mathbf{z}$, and only case (d) is active, giving $(1-\delta)^t$. In general, an adversary deviating on multiple relations is detected by any one of the corresponding checks, so the bound $4(1-\delta)^t$ covers all strategies without further case analysis.

Combining the bounds. Summing the three error terms:

$$\Pr[\mathsf{Acc} \mid \mathbf{AB} \neq \mathbf{C}] \;\le\; \epsilon_{\mathsf{bind}}(\lambda) + \frac{k-1}{|\mathbb{F}|} + 4(1-\delta)^t.$$

Concrete parameters for $\lambda = 128$ bits. Set $|\mathbb{F}| \geq 2^{256}$ so that $(k-1)/|\mathbb{F}| \leq 2^{-128}$ for all practical k. Set $t = \lceil 128/\log_2(1/(1-\delta)) \rceil$ so that $4(1-\delta)^t \leq 4 \cdot 2^{-128} < 2^{-126}$. Under standard collision-resistance assumptions, $\epsilon_{\text{bind}} \leq 2^{-128}$. The total soundness error is therefore at most $3 \cdot 2^{-126} < 2^{-124}$, which is negligible in $\lambda = 128$. This completes the proof of Theorem 5.1. □

Remark 1 (Knowledge Soundness). *The protocol FastMatMul can be extended to a full* argument of knowledge, *meaning one can efficiently extract a valid witness* $(\mathbf{A}, \mathbf{B}, \mathbf{C})$ *with* $\mathbf{AB} = \mathbf{C}$ *from any* PPT *prover* $\mathcal{P}^*$ *that makes the verifier accept with non-negligible probability. The extraction proceeds in two stages. First, the knowledge extractor of the underlying SNARK (invoked via black-box rewind on the inner proof* π*) yields the SNARK witness* $(\mathbf{x}, \mathbf{y}, \mathbf{z}, \{\mathsf{Enc}_\chi^{(j)}, \pi_{\chi,j}\}_{j\in I, \chi\in\mathcal{S}})$. *Second, by rewinding* $\mathcal{P}^*$ *on* k *distinct Freivalds challenges* $r_1, \ldots, r_k \in \mathbb{F}$ *while keeping the initial commitment phase fixed, the extractor collects* k *evaluations* $\mathbf{x}_{r_i} = \mathbf{r}_i\mathbf{A}$ *of the polynomial-in-r determined by the committed* $\mathbf{A}$*. Solving the resulting Vandermonde system over* $\mathbb{F}$ *recovers* $\mathbf{A}$ *exactly; an analogous argument recovers* $\mathbf{B}$ *and* $\mathbf{C}$*. Since the Merkle tree is computationally binding, the extracted matrices are consistent with the committed roots. The full formal proof follows standard rewinding techniques and is deferred to the full version of this work.*

6 Implementation and Evaluation

6.1 Implementation Setup

We implement FastMatMul in Rust. The SNARK backend is a transparent, linear-time-verifiable proof system using Merkle-tree-based commitments; the large proof sizes observed in Table 3 (≈ 1.76–96 MB) are characteristic of such schemes, which trade proof size for transparent setup and avoid a structured reference string. For encoding, we evaluate two configurations:

- **Linear-time encodable code** (used in Table 3). We instantiate C with an expander-based code [11] of constant rate and relative distance δ. The encoding constraints inside the SNARK circuit are $O(k)$, yielding a total circuit size of $O(tk)$.
- **Constant-rate code** (used in Table 4). We sweep over code rates $\rho \in \{1/2, 1/4\}$ to characterize the encoding constraint overhead $O(k(n-k)) = O(k^2/\rho)$, which dominates for large k under general codes.

We compare FastMatMul against a direct SNARK-in-circuit implementation of Freivalds' algorithm [8], where the vector-matrix products $\mathbf{rA}$, $\mathbf{xB}$, and $\mathbf{rC}$ are fully arithmetized, incurring $O(k^2)$ R1CS constraints. The Freivalds baseline uses a pairing-based SNARK backend (consistent with its constant 0.002 s verification time). All experiments use matrices over a 256-bit prime field with k ranging from 2^5 to 2^{13}.

6.2 Constraint Count Comparison

Table 3 and Figure 2 report constraint counts for both protocols. The asymptotic gap between the two approaches is clearly visible. For Freivalds, constraints grow by a factor of approximately 4× with each doubling of k—a signature of $O(k^2)$ scaling—while FastMatMul constraints grow by approximately 2×, confirming $O(k)$ scaling under the linear-time encodable code.

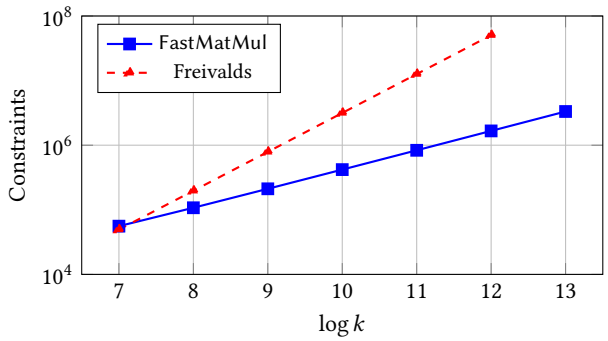

Figure 2: Constraint count comparison on a log scale. FastMatMul exhibits $O(k)$ growth (slope ≈ 1), while Freivalds exhibits $O(k^2)$ (slope ≈ 2). Freivalds exceeds memory at $\log k = 13$.

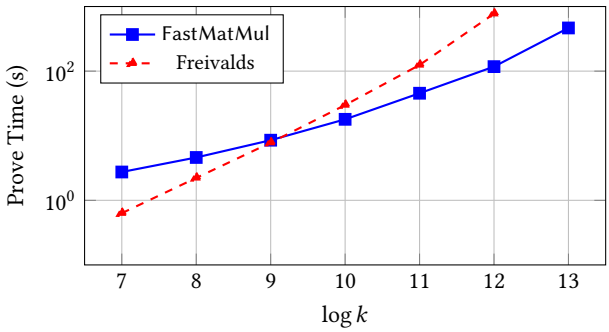

Figure 3: Proving time comparison on a log scale. FastMatMul becomes faster than Freivalds at $\log k = 10$ ($k = 1{,}024$), with the gap widening to 6.6× at $\log k = 12$.

The crossover in absolute constraint count occurs at $k = 256$ ($\log k = 8$): below this threshold, Freivalds' $O(k^2)$ constant is sufficiently small that its circuit is marginally smaller (49,481 vs. 55,828 at $\log k = 7$); above it, FastMatMul is consistently more compact. At $k = 4{,}096$ ($\log k = 12$), FastMatMul uses 1,664,106 constraints versus 50,491,721 for Freivalds—a 30× reduction. At $k = 8{,}192$ ($\log k = 13$), Freivalds exhausts available memory and cannot be run, while FastMatMul completes with 3,324,484 constraints. This demonstrates that FastMatMul extends the practical range of SNARK-based matrix multiplication verification by at least one order of magnitude in matrix dimension.

6.3 Proving Time

Figure 3 and Table 3 show the prove-time crossover between $\log k = 9$ ($k = 512$) and $\log k = 10$ ($k = 1{,}024$). At $\log k = 9$, the two protocols are roughly on par (8.5 s for FastMatMul vs. 7.8 s for Freivalds). At $\log k = 10$, FastMatMul is already 1.7× faster (17.9 s vs. 29.8 s), and the advantage compounds rapidly as k grows: at $\log k = 11$, the speedup is 2.7× (45.5 s vs. 124.5 s); at $\log k = 12$, it reaches 6.6× (117.4 s vs. 773.0 s). At $\log k = 13$, Freivalds is infeasible and FastMatMul completes in 465.7 s.

Table 3: Performance comparison of FastMatMul and the SNARK-in-circuit Freivalds baseline. Constraint counts confirm $O(k)$ vs. $O(k^2)$ scaling. "—" indicates the experiment could not be completed (out of memory or missing data). FastMatMul uses a transparent SNARK backend; Freivalds uses a pairing-based backend.

$\log k$	Constraints		Prove Time (s)		Verify Time (s)		Proof Size (MB)	
	FastMatMul	Freivalds	FastMatMul	Freivalds	FastMatMul	Freivalds	FastMatMul	Freivalds
5	—	3,401	—	—	—	0.002	—	—
6	—	12,617	0.197	—	—	0.002	—	—
7	55,828	49,481	2.745	0.632	0.226	0.002	1.76	—
8	107,546	196,937	4.606	2.223	0.268	0.002	3.28	—
9	211,370	788,297	8.528	7.807	0.348	0.002	6.30	—
10	419,012	3,155,273	17.947	29.835	0.485	0.002	12.32	—
11	833,914	12,620,105	45.468	124.545	0.783	0.002	24.34	—
12	1,664,106	50,491,721	117.393	772.935	1.370	0.002	48.36	—
13	3,324,484	—	465.661	—	2.972	—	96.38	—

We note that for small k ($\log k \leq 9$), Freivalds proves faster due to two factors: (i) the per-constraint overhead of the transparent SNARK backend used by FastMatMul is higher than that of the pairing-based backend used by Freivalds; and (ii) the $O(k)$ constant in FastMatMul's circuit includes Merkle-path hashing, which contributes fixed overhead per query. For large k, these constants are dominated by the circuit size difference, and FastMatMul becomes decisively faster.

6.4 Verification Time and Proof Size

A trade-off of FastMatMul's transparent SNARK backend is that verification time and proof size are larger than the pairing-based Freivalds baseline. Freivalds verification is constant at 0.002 s for all tested k—consistent with constant-time pairing-based SNARK verification. FastMatMul's verification time grows from 0.226 s at $\log k = 7$ to 2.972 s at $\log k = 13$, approximately doubling with each doubling of k. This growth reflects the increase in public input size: the public statement x includes the t queried column openings, whose total size scales as $O(tk)$.

Proof sizes for FastMatMul grow from 1.76 MB ($\log k = 7$) to 96.38 MB ($\log k = 13$), roughly doubling with each unit increase in $\log k$—consistent with the $O(k)$ witness size dominated by the opened codeword columns and their Merkle authentication paths. Freivalds proof sizes are negligible (sub-kilobyte for pairing-based SNARKs) but are reported as "–" in Table 3 since those experiments could not be fully completed beyond $\log k = 6$.

We remark that the proof size and verification time of FastMatMul can be improved by replacing the Merkle-based commitment with an algebraic vector commitment scheme (e.g., KZG [12]), which reduces proof size to $O(\lambda)$ and verification to $O(1)$ at the cost of a structured setup. We leave this combination as future engineering work.

6.5 Encoding Cost Analysis

Table 4 reports the precomputation and encoding times under general constant-rate codes for $\log k \in [10, 20]$ and rates $\rho \in \{1/2, 1/4\}$. Two observations are noteworthy.

Encoding is fast. Even at $\log k = 20$ ($k = 2^{20} \approx 10^6$), encoding completes in 0.623 s (rate 1/2) and 0.680 s (rate 1/4), and precomputation takes at most 0.220 s. These costs are negligible relative to the native $O(k^3)$ matrix multiplication, confirming that encoding does not introduce a new bottleneck in the prover's pipeline.

Rate vs. constraint trade-off. The encoding constraints—which contribute $O(k(n-k)) = O(k^2(1-\rho)/\rho)$ to the circuit—grow roughly as 2× when the rate is halved, as expected. At $\log k = 16$, rate 1/2 yields 328,558 encoding constraints while rate 1/4 yields 591,578 constraints (1.8× more). For the Merkle-hash component (fold and path checks), the constraint count is $O(tk)$ and appears in the FastMatMul column of Table 3; the encoding constraints in Table 4 represent an *additional* overhead incurred when using general codes instead of linear-time encodable codes.

When the encoding constraint cost is acceptable (i.e., when k^2 fits within the SNARK's capacity), rate 1/2 represents a good default, offering a balanced trade-off between code distance δ (which determines the soundness error per query) and encoding constraint overhead. For applications demanding a higher soundness level per query (larger δ), a lower rate with more queries may be preferable.

6.6 Summary

Table 3 and Table 4 together establish the following conclusions about FastMatMul.

Constraint efficiency. FastMatMul achieves $O(k)$ circuit growth (under linear-time encodable codes), versus $O(k^2)$ for Freivalds-based SNARK arithmetization. The constraint advantage reaches 30× at $k = 4{,}096$ and grows without bound. Freivalds is infeasible beyond $k = 4{,}096$ under the tested configuration; FastMatMul scales to at least $k = 8{,}192$.

Proving time. FastMatMul outperforms Freivalds in prove time for $k \geq 1{,}024$ ($\log k = 10$), with speedups of 1.7×–6.6× in the range $k \in [1{,}024,\ 4{,}096]$. For smaller matrices ($k < 512$), Freivalds proves faster due to its lower per-constraint overhead under a pairing-based backend.

Verification and proof size. FastMatMul's verification is slower (0.2–3 s) and proofs are larger (1.8–96 MB) than Freivalds. These costs reflect the transparency of the backend and can be improved

Table 4: Precomputation and encoding cost under general constant-rate codes. ρ: code rate; Constraint: number of encoding-only R1CS constraints (additional to the fold/Merkle constraints in Table 3).

$\log k$	ρ	Precomp. (s)	Encoding (s)	Constraints
10	1/2	0.000	0.001	5,134
10	1/4	0.000	0.001	9,244
12	1/2	0.000	0.002	20,536
12	1/4	0.001	0.003	36,974
14	1/2	0.002	0.008	82,140
14	1/4	0.005	0.010	147,896
16	1/2	0.007	0.035	328,558
16	1/4	0.012	0.040	591,578
18	1/2	0.024	0.126	1,314,228
18	1/4	0.049	0.152	2,366,310
20	1/2	0.104	0.623	5,256,908
20	1/4	0.220	0.680	9,465,240

by switching to an algebraic commitment scheme with a structured setup.

Practical recommendation. For verifiable ML inference tasks involving matrices with $k \geq 1{,}024$—the regime of practical interest for modern Transformer layers—FastMatMul is the preferred choice. Its combination of linear circuit growth, competitive proving time, and constant verifier work makes it well-suited for deployment scenarios where the prover (inference server) bears the computation cost and the verifier (client) requires only a succinct check.

7 Related Work

We position our work within four lines of research: verifiable ML inference, dedicated matrix multiplication verification, code-based proof systems, and the sumcheck approach.

Verifiable ML inference. SafetyNets [9] was among the first to propose interactive proofs for deep neural networks, employing the sumcheck protocol to verify matrix multiplications layer by layer. zkCNN [14] extended this to convolutional layers by reducing convolution to polynomial multiplication and leveraging the GKR protocol [10]. vCNN [13] further optimized the verification of quantized convolution operations. Mystique [18] and zkML [4] have explored practical zero-knowledge proof systems for general ML inference, including Transformer architectures. In all of these systems, matrix multiplication is the dominant cost center. Our protocol is complementary: it provides a drop-in replacement for the matrix multiplication verification subroutine that reduces the SNARK circuit size from $O(k^2)$ to $O(tk)$. Composing FastMatMul across the layers of a neural network is a natural extension that we outline in Section 8.

Dedicated matrix multiplication verification. zkMatrix [5] proposed a batched short-proof protocol for committed matrix multiplication using customized polynomial commitments, achieving proof sizes of $O(\sqrt{k})$ group elements. DualMatrix [6] combined polynomial evaluation and inner-product arguments to reduce the prover cost, though the prover still performs $O(k^2)$ work inside the proof system. zkVC [20] introduced Constraint-Reduced Polynomial Circuits (CRPC) and Prefix-Sum Query (PSQ) techniques to reduce the R1CS constraint count, reporting a 12× prover speedup and demonstrating applicability to Transformer inference. Bennett et al. [3] explored the use of coding theory for matrix multiplication verification in an information-theoretic setting, establishing theoretical connections between code distance and verification complexity. Our work shares the coding-theoretic perspective of [3] but differs in two key respects: (i) we operate in the computational setting with Merkle commitments and SNARKs, yielding a *non-interactive* protocol with succinct proofs; and (ii) we provide a concrete implementation with experimental evaluation, demonstrating practical gains over the Freivalds baseline. Relative to zkMatrix, DualMatrix, and zkVC, our approach avoids polynomial commitment machinery entirely, relying instead on hash-based commitments that require no trusted setup.

Code-based proof systems. Ligero [1] pioneered the use of interleaved Reed–Solomon codes to construct transparent zero-knowledge proofs with $O(\sqrt{n})$ proof size. Brakedown [11] introduced expander-based linear codes as a drop-in replacement for polynomial commitments, achieving linear-time prover complexity for general-purpose R1CS. Orion [19] combined code-based commitments with the sumcheck protocol. FRI [2] (Fast Reed–Solomon Interactive Oracle Proofs of Proximity) established the foundation for proximity testing of Reed–Solomon codes, underpinning the STARK family of proof systems. Our work draws on the code-based commitment philosophy of Brakedown but applies it to the *structured* setting of matrix multiplication, where the linearity of encoding interacts directly with the Freivalds reduction to yield a circuit-size improvement beyond what a generic code-based SNARK achieves. Specifically, while Brakedown provides a general $O(n)$-prover SNARK for any circuit of size n, naïvely feeding the $O(k^2)$ Freivalds circuit into Brakedown still yields an $O(k^2)$ prover. FastMatMul's contribution is to *restructure the problem itself*—via the relation chain $\mathcal{R}_{\text{orig}} \to \mathcal{R}_{\text{Freivalds}} \to \mathcal{R}_{\text{SNARK}}$—so that the circuit presented to the SNARK is only $O(tk)$.

Sumcheck-based approaches. The sumcheck protocol [10] is a powerful interactive proof technique for verifying the evaluation of multivariate polynomials. When applied to matrix multiplication, it reduces the verifier's work to evaluating multilinear extensions of the input matrices at a random point, which requires $O(k^2)$ field operations for the verifier (or $O(k^2)$ oracle queries). SafetyNets [9] and zkCNN [14] use this approach. Unlike sumcheck-based methods, our proximity-based checks are purely combinatorial: they compare codeword coordinates via inner products rather than evaluating polynomial extensions, avoiding the algebraic overhead of sumcheck rounds. Moreover, our protocol naturally interfaces with any SNARK backend (the local checks are expressed as R1CS or PlonK constraints), whereas sumcheck-based systems typically require a specialized verifier or a dedicated GKR-to-SNARK compilation step.

8 Conclusion and Future Work

We have presented FastMatMul, a verifiable computation protocol for matrix multiplication that achieves a SNARK circuit size of $O(tk)$, where t is a small constant determined by the security parameter and k is the matrix dimension. The protocol combines Freivalds' randomized reduction with proximity testing over linear error-correcting codes, cleanly decoupling the expensive matrix arithmetic (performed outside the SNARK by the prover) from the lightweight algebraic checks (arithmetized inside the SNARK). We proved that FastMatMul achieves perfect completeness and computational soundness with error $\epsilon_{\text{bind}} + (k-1)/|\mathbb{F}| + 4(1-\delta)^t$.

Our experimental evaluation demonstrates that the theoretical advantages translate into concrete gains: FastMatMul reduces the constraint count by up to 30× at $k = 4{,}096$ relative to a direct Freivalds SNARK baseline, achieves 1.7–6.6× proving-time speedups for $k \geq 1{,}024$, and continues to scale at $k = 8{,}192$ where the baseline exhausts memory. The off-circuit encoding cost is negligible (under 1 s at $k = 2^{20}$), confirming that the code-based approach does not introduce a new bottleneck.

Future directions. Several avenues remain. First, while our protocol targets a single matrix multiplication, deep learning inference involves *chains* of matrix multiplications interleaved with non-linear activations. Extending FastMatMul to handle layered computations—by chaining the commitment outputs of one layer as inputs to the next—is a natural step toward fully verifiable neural network inference. Second, the encoding constraints, which contribute $O(k^2)$ under general constant-rate codes, could potentially be deferred to a separate proximity test (e.g., FRI [2]), further reducing the circuit. Third, replacing the Merkle-based commitment with an algebraic vector commitment (e.g., KZG) would reduce proof sizes from megabytes to kilobytes at the cost of a structured setup; evaluating this trade-off in practice is an important engineering task. Finally, exploring *batched verification*—where multiple matrix multiplications share a single encoding and commitment phase—could yield significant amortized cost savings in practical deployments.

References

[1] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. 2017. Ligero: Lightweight Sublinear Arguments Without a Trusted Setup. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. ACM, New York, NY, USA, 2087–2104.

[2] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. 2018. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 14:1–14:17.

[3] Huck Bennett, Karthik Gajulapalli, Alexander Golovnev, and Evelyn Warton. 2023. Matrix multiplication verification using coding theory. *arXiv preprint arXiv:2309.16176* (2023).

[4] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. 2024. Zkml: An optimizing system for ml inference in zero-knowledge proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems*. ACM, New York, NY, USA, 560–574.

[5] Mingshu Cong, Tsz Hon Yuen, and Siu-Ming Yiu. 2024. zkmatrix: Batched short proof for committed matrix multiplication. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. ACM, New York, NY, USA, 289–305.

[6] Mingshu Cong, Tsz Hon Yuen, and Siu Ming Yiu. 2026. Dualmatrix: conquering zkSNARK for large matrix multiplication. *Cybersecurity* 9, 1 (2026), 126.

[7] Amos Fiat and Adi Shamir. 1987. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology – CRYPTO '86*. Springer, Berlin, Heidelberg, 186–194.

[8] Rūsiņš Freivalds. 1977. Probabilistic Machines Can Use Less Running Time. In *Information Processing 77 (Proceedings of IFIP Congress 77)*. North-Holland, Amsterdam, The Netherlands, 839–842.

[9] Zahra Ghodsi, Tianyu Gu, and Siddharth Garg. 2017. Safetynets: Verifiable execution of deep neural networks on an untrusted cloud. *Advances in Neural Information Processing Systems* 30 (2017).

[10] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. 2015. Delegating computation: Interactive proofs for muggles. *J. ACM* 62, 4 (2015), 1–64.

[11] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad S. Wahby. 2023. Brakedown: Linear-time and field-agnostic SNARKs for R1CS. In *Advances in Cryptology – CRYPTO 2023*. Springer, Cham, 193–226.

[12] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. 2010. Constant-Size Commitments to Polynomials and Their Applications. In *Advances in Cryptology – ASIACRYPT 2010*. Springer, Berlin, Heidelberg, 177–194.

[13] Seunghwa Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. 2024. vcnn: Verifiable convolutional neural network based on zk-snarks. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2024), 4254–4270.

[14] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. Zkcnn: Zero knowledge proofs for convolutional neural network predictions and accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. ACM, New York, NY, USA, 2968–2985.

[15] Ralph C. Merkle. 1989. A Certified Digital Signature. In *Advances in Cryptology – CRYPTO '89*. Springer, Berlin, Heidelberg, 218–238.

[16] Torben Pryds Pedersen. 1991. Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing. In *Advances in Cryptology – CRYPTO '91*. Springer, Berlin, Heidelberg, 129–140.

[17] Jacob T. Schwartz. 1980. Fast probabilistic algorithms for verification of polynomial identities. *J. ACM* 27, 4 (1980), 701–717.

[18] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. 2021. Mystique: Efficient conversions for {Zero-Knowledge} proofs with applications to machine learning. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Berkeley, CA, USA, 501–518.

[19] Tiancheng Xie, Yupeng Zhang, and Dawn Song. 2022. Orion: Zero Knowledge Proof with Linear Prover Time. In *Advances in Cryptology – CRYPTO 2022*. Springer, Cham, 299–328.

[20] Yancheng Zhang, Mengxin Zheng, Xun Chen, Jingtong Hu, Weidong Shi, Lei Ju, Yan Solihin, and Qian Lou. 2025. zkvc: Fast zero-knowledge proof for private and verifiable computing. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, IEEE, New York, NY, USA, 1–7.

[21] Richard Zippel. 1979. Probabilistic algorithms for sparse polynomials. In *Symbolic and Algebraic Computation (EUROSAM '79)*. Springer, Berlin, Heidelberg, 216–226.

A Probabilistic Verification of Reed–Solomon Codewords

A.1 Problem Definition

Let $\mathbb{F}$ be a finite field. Consider a message vector $\mathbf{m} = (a_0, \ldots, a_{k-1}) \in \mathbb{F}^k$ and a received word $\mathbf{c} = (c_0, \ldots, c_{n-1}) \in \mathbb{F}^n$, where $n \geq k$. The evaluation domain is defined by distinct points $S = \{w^0, w^1, \ldots, w^{n-1}\} \subseteq \mathbb{F}$.

We define two polynomials:

(1) **Message polynomial** $f(X) = \sum_{i=0}^{k-1} a_i X^i$ with $\deg(f) < k$.
(2) **Interpolation polynomial** $g(X)$: the unique polynomial of degree less than n satisfying $g(w^i) = c_i$ for all $i = 0, \ldots, n-1$.

Goal. Efficiently verify whether $\mathbf{c}$ is a valid Reed–Solomon encoding of $\mathbf{m}$, i.e., whether $f(X) \equiv g(X)$.

A.2 Protocol: Randomized Consistency Check

Instead of re-encoding $\mathbf{m}$ (which costs $O(k \log k)$), we apply the Schwartz–Zippel lemma (Lemma 3.2) to check consistency at a random point $z \in \mathbb{F}$.

A.3 Soundness Analysis

By the Schwartz–Zippel lemma, if $f(X) \not\equiv g(X)$, the polynomial $h(X) = f(X) - g(X)$ is non-zero with degree at most $\max(\deg f, \deg g) < n$. The probability that a uniformly random $z \in \mathbb{F} \setminus S$ satisfies

Algorithm 1 Probabilistic Verification of RS Codeword

Require: Message coefficients $\{a_i\}_{i=0}^{k-1}$, codeword values $\{c_i\}_{i=0}^{n-1}$, domain $\{w^i\}_{i=0}^{n-1}$

Verifier samples $z \leftarrow\!\!\$\ \mathbb{F} \setminus S$.

Evaluate $f(z)$**.**

$v_1 \leftarrow f(z) = \sum_{i=0}^{k-1} a_i z^i$ (via Horner's method, $O(k)$ operations)

Evaluate $g(z)$ **via Barycentric Interpolation.**

Precompute barycentric weights λ_i for the domain S.

$$v_2 \leftarrow g(z) = \frac{\displaystyle\sum_{i=0}^{n-1} \frac{\lambda_i}{z - w^i}\, c_i}{\displaystyle\sum_{i=0}^{n-1} \frac{\lambda_i}{z - w^i}}$$

($O(n)$ field operations via barycentric interpolation)

if $v_1 = v_2$ **then**

  return Accept (valid codeword with high probability)

else

  return Reject (invalid codeword)

end if

$f(z) = g(z)$ is:

$$\Pr_{z \leftarrow\!\!\$\, \mathbb{F}}[f(z) = g(z)] \leq \frac{n-1}{|\mathbb{F}|}.$$

For a sufficiently large field ($|\mathbb{F}| \gg n$), this error probability is negligible.